Podsetnik za konsultacije

Samo za mene — šta da kažem, i odgovori na verovatna pitanja

Kako da vodim prezentaciju

Ukupno 27 slajdova, oko 15–20 minuta. Glavna nit priče: **prvi model je imao 97% i nije radio — otkrio sam zašto, izabrao bolji skup podataka, i ovaj put dokazao da model gleda grinju.**

Slajd	Šta kažem (jedna–dve rečenice)
1–2	Tema i pregled. Izabrani model ima 93,1% na snimcima koje nikad nije video.
3	Pregledao sam 5 javnih skupova. Ključni uslov: oznaka mora opisivati **pčelu na slici**.
4–5	Prvi skup, BeeImage: 97% na testu. Ali u aplikaciji — pčela sa dve grinje → „zdrava“ 99,97%.
6	Uzrok: oznaka je po **snimku**, ne po pčeli. Varoa = 5 snimaka, i isti snimak je bio i u treningu i u testu.
7	Dokaz: iz samih uglova slike (bez pčele) model „vidi“ varou u 74% slučajeva. Naučio je pozadinu. To se zove *shortcut learning*.
8	Iz toga četiri uslova za novi skup.
9–10	VarroaDataset: svaka grinja ručno označena, ista podloga, podela po snimku od autora.
11	Transfer learning: ImageNet mreža, nova glava sa 2 izlaza, prvo glava pa cela mreža. Letterbox da se ne odseče zadak.
12	Prvi protokol (kao kod BeeImage-a) nije naučio grinju — val F1 0,665. Fine-tuning cele mreže: 0,934. Birao sam **samo na validacionom** skupu.
13	Krive učenja — stabilno, bez preprilagođavanja.
14–16	Rezultati: ResNet50 najbolji po svim metrikama koje ste tražili. Matrice konfuzije, ROC.
17	Analiza grešaka: najviše promašaja kad je grinja skrivena ispod krila.
18–19	**Najvažniji slajd:** kad prekrijem grinju, model menja mišljenje (3,9% ostaje „varoa“); kad prekrijem drugo mesto — ne menja (98,2%). Model gleda grinju.
20	Iz pozadine novi model kaže „varoa“ u ~1% slučajeva (stari 74%). Grad-CAM.
21	Na drugoj kameri (BeeDataset) model razlikuje klase (AUC), ali je previše oprezan — rešivo pragom.
22	Terenske fotografije: ona koja je oborila stari model sada je tačna.

Slajd	Šta kažem (jedna–dve rečenice)
23–24	Zašto ResNet50, i poređenje starog i novog modela.
25	Integracija (backend i frontend je već videla — kratko).
26	Ograničenja i sledeći koraci: terenski test skup.

Brojevi koje treba da znam

VarroaDataset	13.509 slika (9.562 zdrave, 3.947 varoa), 13 snimaka
Podela	train 8.225 · val 1.876 · test 3.408 (2.466 zdravih, 942 varoa)
ResNet50, test	tačnost **0,931**, preciznost 0,915, odziv 0,912, **F1 0,913**, odziv varoe 0,868, AUC 0,975
ResNet50, matrica	2.355 tačno zdravih · **111 lažnih uzbuna** · **124 propuštene** · 818 tačno varoa
MobileNetV3 / EfficientNetB0	F1 0,901 / 0,901 · odziv varoe 0,876 / 0,842
Brisanje grinje (ResNet50)	grinja prekrivena → 3,9% „varoa“ · drugo mesto → 98,2%
BeeImage stari model	97,0% na testu, iz uglova 74% „varoa“
Izbor protokola (val F1)	A poslednji blok 0,665 · **B cela mreža 0,934** · C veća slika 0,943 (4,7× sporije)
BeeDataset (ResNet50)	AUC 0,869, odziv varoe 0,305 na pragu 0,5 (MobileNetV3: 0,949 / 0,476)
Brzina (ONNX, CPU)	MobileNetV3 2,6 ms · EfficientNetB0 4,5 ms · ResNet50 11,9 ms
Veličina	17,1 MB · 16,6 MB · 94,2 MB
Trening (Apple M5)	MobileNetV3 20 min · EfficientNetB0 25 min · ResNet50 40 min

Pitanja o skupu podataka

Zašto niste ostali na BeeImage-u — to je najpoznatiji skup? Zato što oznaka u njemu opisuje stanje košnice u trenutku snimanja, ne pčelu na slici. Sve pčele sa jednog snimka dobiju istu oznaku, i na većini slika „varoe“ grinja se ne vidi. Model je zato naučio pozadinu snimka — to sam i dokazao testom uglova.

Zašto VarroaDataset? Jedini javni skup koji ispunjava sva tri uslova: oznaka po pčeli (sa ručno označenom grinjom), ista podloga za obe klase i podela po snimku koju su dali autori. Uz to je veći (13.509 slika).

Šta je oznaka 3? Nije opisana u dokumentaciji. Autori je u zvaničnoj statistici broje kao zaraženu, sve takve slike imaju označenu grinju, a vizuelno je grinja delimično skrivena. Zato je računam kao varou, a rezultat prikazujem i odvojeno — tamo model najviše greši.

Zašto samo dve klase, a ne šest kao u BeeImage-u? Nijedan javni skup nema pouzdane oznake po pojedinačnoj pčeli za druga stanja. Bolje dve klase koje model stvarno prepoznaje nego šest koje ne prepoznaje. Varoa je najvažniji zdravstveni problem.

Šta je BeeDataset i zašto ga koristite? Drugi javni skup (BeeAlarmed, Nemačka), druga kamera, zelena podloga. Model ga nikad nije video, pa meri kako se model ponaša na drugačijem snimanju — bliže stvarnoj upotrebi od testa na istom skupu.

Pitanja o mašinskom učenju

Šta je transfer learning? Uzimam mrežu već istreniranu na ImageNet-u (1,28 miliona slika, 1.000 klasa). Ona već zna da prepozna ivice, teksture, oblike. Zamenim samo poslednji sloj (1.000 → 2 klase) i doučim je na mom skupu. Tako je dovoljno nekoliko hiljada slika umesto miliona.

Zašto prvo samo glava, pa cela mreža? Nova glava je na početku nasumična. Da odmah treniram celu mrežu, nasumični gradijenti bi pokvarili naučene slojeve. Zato 3 epohe samo glava, pa 15 epoha cela mreža sa 10× manjom stopom učenja.

Zašto cela mreža, a ne samo poslednji blok (kao kod BeeImage-a)? Probao sam oba na validacionom skupu. Sa samo poslednjim blokom model nije naučio ni trening skup (74% tačnosti na treningu, val F1 0,665). Grinja je sitna (~25 piksela) i ne liči ni na šta iz ImageNet-a, pa i niži slojevi moraju da se prilagode. Sa celom mrežom: 0,934.

Zar podešavanje protokola nije „varanje“? Ne, jer sam birao isključivo na **validacionom** skupu (drugi snimci), a **test** skup sam koristio tačno jednom, na kraju. Isti protokol je primenjen na sve tri arhitekture, pa je poređenje pod istim uslovima.

Šta je letterbox? Slike su uspravne (160×280). Standardna priprema iseca centralni kvadrat i odbacuje ~45% visine — glavu i zadak. Letterbox smanji celu sliku i dopuni do kvadrata, ništa se ne odseca.

Šta je augmentacija? Svaka slika se pri svakom prolazu nasumično rotira, preslika, pomeri, promeni joj se osvetljenje i boja. Model tako nikad ne vidi dvaput istu sliku i bolje generalizuje.

Kako ste rešili disbalans klasa (2,4 : 1)? Težinska funkcija greške — greška na zaraženoj pčeli košta više (1,61 naspram 0,73). I biram model po macro F1, a ne po tačnosti, jer bi model koji uvek kaže „zdrava“ imao 69% tačnosti.

Accuracy, precision, recall, F1 — šta je šta?

- *Accuracy* — udeo tačnih odluka.
- *Precision (varoa)* — kad model kaže „varoa“, koliko često je u pravu (malo lažnih uzbuna).
- *Recall (varoa)* — od svih zaraženih, koliko je model pronašao (malo propuštenih).
- *F1* — harmonijska sredina precision i recall. *Macro* — prosek preko obe klase.

Šta je ROC AUC? Koliko dobro model razdvaja zdrave od zaraženih, za sve moguće pragove odluke. 1,0 savršeno, 0,5 nasumično.

Kako znate da model gleda grinju? Test brisanja grinje: na zaraženoj pčeli prekrijem grinju bojom okoline — procena „varoa“ ostaje kod 3,9%. Kada isto toliko prekrijem drugo mesto na pčeli — ostaje

kod 98,2%. Odluka zavisi baš od grinje. To je uzročni dokaz. Dodatno: iz same pozadine model kaže „varoa“ u ~1% slučajeva, a Grad-CAM pokazuje gde gleda.

Šta je Grad-CAM i zašto ResNet50 ima samo 46%? Mapa koja boji delove slike koji su najviše uticali na odluku. Kod ResNet50 mapa je šira i obuhvata okolni deo tela, pa najjača tačka često nije baš u pravougaoniku grinje. Ali test brisanja jasno pokazuje da je grinja presudna — Grad-CAM je samo opisni alat.

Zašto je tačnost pala sa 97% na 93%? Stari broj je bio lažno visok: test je sadržao iste snimke kao trening, a model je učio pozadinu. Novih 93% je na snimcima koje model nikad nije video, i model dokazano gleda grinju. Niži, ali pošten broj.

Zašto ResNet50, ako je MobileNetV3 bolji na BeeDataset-u i brži? ResNet50 je najbolji po svim metrikama koje ste tražili, ima najmanje lažnih uzbuna i tačno prepoznaje terensku fotografiju. Brzina (12 ms) je za server nebitna. Ali razlike su oko 1 procentnog poena — da se model izvršava na telefonu, izabrao bih MobileNetV3. Zamena modela je zamena jednog fajla.

Da li su razlike između modela značajne? Ne mogu to da tvrdim — svaki model je treniran jednom. Za to bi trebalo ponoviti trening sa više semena. To je navedeno u ograničenjima.

Zašto PyTorch, a ne TensorFlow/Keras? Priprema slike je vezana za težine modela (`weights.transforms()`), pa ne mogu slučajno da poredim modele pod različitim uslovima. Jedna petlja treninga za sve tri arhitekture. Direktan izvoz u ONNX. Nativna podrška za grafiku Apple čipa (MPS).

Da li je model spreman za praksu? Za pomoćnu procenu — da, ako je fotografija jedne pčele izbliza. Na drugačijim snimcima je previše oprezan (propušta deo zaraza). Sledeći korak je terenski test skup sa 20–40 fotografija i podešavanje praga odluke. Važno: model ocenjuje pčelu, ne celu košnicu — pčela bez vidljive grinje može biti iz zaražene košnice.

Pitanja o vezi sa backend-om

Kako ste povezali Python model sa .NET backend-om? Model radi u zasebnom **Python servisu napravljenom u FastAPI-ju**. ASP.NET backend mu šalje fotografiju preko HTTP-a i dobija JSON sa procenom. Angular ne razgovara direktno sa Python-om — sve ide preko .NET backend-a, koji proverava prava pristupa i čuva rezultat u bazi.

Šta je FastAPI? Python biblioteka za pravljenje REST API-ja — po ulozi slična ASP.NET Minimal API-ju. Pokreće se serverom `uvicorn`. Servis ima dve rute: `GET /health` i `POST /predict`.

Kako model radi unutar servisa? Model sam iz PyTorch-a izvezao u format **ONNX** (otvoreni format za neuronske mreže). Servis ga učitava bibliotekom **onnxruntime** — jednom, pri pokretanju. Na svaki zahtev: priprema slike (letterbox + normalizacija) → model → softmax → JSON. Servis zato ne zavisi od PyTorch-a.

Zašto ne direktno u .NET-u (ML.NET / ONNX Runtime za C#)? Mogao bih, ali bih morao ponovo da napišem pripremu slike u C#. Svako sitno odstupanje od treninga tiho kvari rezultate, bez greške. Ovako je **ista Python funkcija** (`service/preprocessing.py`) u treningu i u servisu. Proverio sam: kroz servis je svih 3.408 test slika dalo **identične** odluke kao originalni model. Plus: model se menja bez ponovnog objavljivanja .NET aplikacije.

Kako .NET poziva servis — šta ste koristili?

- `Program.cs` — registracija *typed HttpClient*-a, adresa iz promenljive okruženja `ML_SERVICE_URL` ili iz `appsettings.json` (`MlService:BaseUrl`, lokalno `http://localhost:8000`):

```
builder.Services.AddHttpClient<IBeeHealthService, BeeHealthService>(client =>
{
    var baseUrl = Environment.GetEnvironmentVariable("ML_SERVICE_URL")
                  ?? builder.Configuration["MlService:BaseUrl"]!;
    client.BaseAddress = new Uri(baseUrl.TrimEnd('/') + "/");
    client.Timeout = TimeSpan.FromSeconds(30);
});
```

- `Services/BeeHealthService.cs` — šalje sliku kao `multipart/form-data`:

```
using var content = new MultipartFormDataContent();
content.Add(new StreamContent(image.OpenReadStream()), "file", image.FileName);
var response = await _httpClient.PostAsync("predict", content);
var result = await response.Content.ReadFromJsonAsync<BeeHealthPredictionResult>();
```

- `BeeHealthPredictionResult` mapira JSON polja servisa (`label`, `confidence`, `all_probs`) atributom `[JsonPropertyName]`.

Šta se dešava ako Python servis ne radi? `BeeHealthService` uhvati grešku konekcije ili isteka vremena i baci `BeeHealthServiceUnavailableException`; kontroler vraća **503** sa porukom „Servis za analizu fotografija trenutno nije dostupan“, a slika se **ne čuva** — prvo se poziva model, pa tek ako uspe, čuva se slika i upisuje rezultat. Ostatak aplikacije radi normalno.

Koje rute ima backend?

Ruta	Opis
`POST /api/hives/{hiveId}/health-checks`	analiza fotografije (multipart, polje `file`)
`GET /api/hives/{hiveId}/health-checks`	istorija pregleda košnice
`DELETE /api/hives/{hiveId}/health-checks/{id}`	brisanje (samo pčelar)

Kontroler: `Controllers/BeeHealthChecksController.cs`. Proverava da košnica pripada korisniku (ili njegovom poslodavcu, za asistente) — tuđa košnica vraća 404. Dozvoljeni formati JPG, PNG, WEBP, do 10 MB.

Gde se čuva rezultat? Nova tabela `BeeHealthChecks` (EF Core migracija `AddBeeHealthChecks`): košnica, URL slike, predviđena klasa, verovatnoća, verovatnoće svih klasa (kolona tipa `jsonb`), verzija modela, vreme i korisnik. Slike čuva `LocalStorageService` u `wwwroot/uploads/` — ranije postojeći `MockStorageService` nije čuvao fajlove, pa sam ga zamenio.

A frontend? Novi tab „Zdravlje pčela“ u modalu košnice (komponenta `hive-health-check`, servis `bee-health.service.ts`). Slika se šalje kao `FormData` kroz postojeći Angular `HttpClient` (JWT se dodaje automatski). Prikazuje procenu u boji, verovatnoće obe klase i istoriju pregleda.

Kako bi se ovo objavilo (deployment)? Tri servisa: Angular, ASP.NET i Python. Za Python servis postoji `service/Dockerfile` (python:3.12-slim, samo onnxruntime — bez PyTorch-a). Backend dobija adresu

servisa kroz `ML_SERVICE_URL`. Za slike bi na hostingu trebalo trajno skladište (npr. Cloudinary) — interfejs `IStorageService` je za to spreman.

Ako pita za kod i rezultate

- GitHub: `github.com/MedzidJasarovic/KosnicaML`
- Notebook-ovi u `notebooks/` — KORAK 1–5, svaki grafik iz prezentacije je tamo
- Detaljan izveštaj: `report/izvestaj_final.pdf`
- Ceo eksperiment se ponavlja jednom komandom: `./run_varroa.sh`